GPU-Accelerated Adaptive Clustering via Local Linearization

Algorithm Specification

November 20, 2025

1 Introduction

This document formalizes the Adaptive Cluster Algorithm. The method recursively partitions a dataset $\mathcal{X} \subset \mathbb{R}^d$ into clusters such that the target function $f : \mathbb{R}^d \to \mathbb{R}^m$ is approximately linear within each cluster. The implementation leverages GPU parallelism by processing all active clusters in broad phases (Evaluation, Splitting) rather than depth-first recursion.

2 Mathematical Notation

Symbol	Description
$\mathcal{X}$	Input dataset, a set of points $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset \mathbb{R}^d$.
$f(\mathbf{x})$	The target vector-valued function $f : \mathbb{R}^d \to \mathbb{R}^m$.
$\mathbf{J}(\mathbf{x})$	The Jacobian matrix of f at $\mathbf{x}$, where $\mathbf{J} \in \mathbb{R}^{m \times d}$.
$\mathcal{C}^{(t)}$	The set of active clusters at iteration t. Each $C_k \in \mathcal{C}^{(t)}$ is a subset of indices.
$\mathbf{c}_k$	The centroid (mean) of cluster C_k.
$N_{\min}$	Minimum number of points required per cluster for statistical stability.
τ_{abs}, τ_{rel}	Absolute and relative error tolerances for linearization.
$\mathcal{L}(\mathbf{x}; \mathbf{c})$	The first-order Taylor approximation of $f(\mathbf{x})$ centered at $\mathbf{c}$.

2.1 Linearity Criterion

For a specific cluster C_k with centroid $\mathbf{c}_k$, we define the linear approximation for any point $\mathbf{x} \in C_k$ as:

$$\mathcal{L}(\mathbf{x}; \mathbf{c}_k) = f(\mathbf{c}_k) + \mathbf{J}(\mathbf{c}_k)(\mathbf{x} - \mathbf{c}_k) \tag{1}$$

A point $\mathbf{x}$ is considered *converged* if the approximation error satisfies:

$$\|f(\mathbf{x}) - \mathcal{L}(\mathbf{x}; \mathbf{c}_k)\|_2 \leq \max\left(\tau_{abs},\, \tau_{rel} \cdot \|f(\mathbf{x})\|_2\right) \tag{2}$$

A cluster C_k is considered converged only if all $\mathbf{x} \in C_k$ satisfy this condition.

2.2 Data Augmentation

Synthetic points are generated for small clusters (where $|C_k| < N_{\min}$), via convex combinations of existing data, while preserving the geometric bounds of the cluster (the convex hull) A.

3 Algorithm Pseudocode

Algorithm 1: Adaptive Clustering with GPU Vectorization

Input : Dataset $\mathcal{X}$, Functions $f, \nabla f$, Initial K_{init}, Tolerances τ_{abs}, τ_{rel}
Output: Set of converged clusters $\mathcal{C}_{final}$

`/* Initialization via k-Means */`
$\mathcal{C}_{active} \leftarrow$ `cuKMeans`$(\mathcal{X}, K_{init})$;
$\mathcal{C}_{final} \leftarrow \emptyset$;
$depth \leftarrow 0$;
while $\mathcal{C}_{active} \neq \emptyset$ ***and*** $depth < MAX_DEPTH$ **do**
  `/* Phase 1: Batch Augmentation & Evaluation */`
  for *each cluster* $C_k \in \mathcal{C}_{active}$ **do in parallel**
    $\tilde{C}_k \leftarrow$ `AugmentPoints`$(C_k, N_{\min})$; `// Ensure min size for stability`
    $\mathbf{c}_k \leftarrow \frac{1}{|\tilde{C}_k|} \sum_{\mathbf{x} \in \tilde{C}_k} \mathbf{x}$;
    Evaluate $f(\mathbf{c}_k)$ and $\mathbf{J}(\mathbf{c}_k)$;
    `/* Check linearity condition for all points in parallel */`
    $\mathcal{I}_{violating} \leftarrow \{\mathbf{x} \in \tilde{C}_k \mid \|f(\mathbf{x}) - \mathcal{L}(\mathbf{x}; \mathbf{c}_k)\| > \text{tol}(\mathbf{x})\}$;
  `/* Phase 2: Filter Converged Clusters */`
  Let $\mathcal{K}_{bad}$ be indices of clusters where $|\mathcal{I}_{violating}| > 0$;
  Let $\mathcal{K}_{good}$ be indices of clusters where $|\mathcal{I}_{violating}| = 0$;
  $\mathcal{C}_{final} \leftarrow \mathcal{C}_{final} \cup \{\tilde{C}_k \mid k \in \mathcal{K}_{good}\}$;
  if $\mathcal{K}_{bad} = \emptyset$ **then**
    break loop;
  `/* Phase 3: Compute Eigenvectors via Power Iteration for Splitting */`
  Initialize eigenvectors $\mathbf{V} \in \mathbb{R}^{|\mathcal{K}_{bad}| \times d}$ randomly;
  for $iter = 1$ ***to*** $PowerIters$ **do**
    for *each* $k \in \mathcal{K}_{bad}$ **do in parallel**
      $\mathbf{X}_k \leftarrow \tilde{C}_k - \mathbf{c}_k$; `// Centered points`
      $\mathbf{v}_k \leftarrow \mathbf{X}_k^T(\mathbf{X}_k \mathbf{v}_k)$; `// Covariance application`
      $\mathbf{v}_k \leftarrow \mathbf{v}_k / \|\mathbf{v}_k\|$; `// Normalize`
  $\mathcal{C}_{next} \leftarrow \emptyset$;
  for *each* $k \in \mathcal{K}_{bad}$ **do in parallel**
    Project points: $p(\mathbf{x}) = \mathbf{x} \cdot \mathbf{v}_k$ for all $\mathbf{x} \in \tilde{C}_k$;
    Sort $\tilde{C}_k$ based on $p(\mathbf{x})$;
    Split $\tilde{C}_k$ into C_{left}, C_{right} at median index;
    $\mathcal{C}_{next} \leftarrow \mathcal{C}_{next} \cup \{C_{left}, C_{right}\}$;
  $\mathcal{C}_{active} \leftarrow \mathcal{C}_{next}$;
  $depth \leftarrow depth + 1$;
return $\mathcal{C}_{final}$

A Data Augmentation via Convex Interpolation

A.1 Vectorized Pair Selection

The GPU implementation requires selecting pairs of distinct indices (i, j) such that $i \neq j$ without branching or rejection sampling. This is achieved using a modular arithmetic strategy.

For a cluster of size S, let indices be in the range $[0, S-1]$. We sample a primary index i and a strictly positive offset δ:

$$i \sim \mathcal{U}\{0, \dots, S-1\} \tag{3}$$

$$\delta \sim \mathcal{U}\{1, \dots, S-1\} \tag{4}$$

The second index j is computed as:

$$j = (i + \delta) \pmod{S} \tag{5}$$

By ensuring $\delta \neq 0$, this guarantees $i \neq j$ deterministically.

A.2 Interpolation

The synthetic point $\mathbf{x}_{syn}$ is generated via linear interpolation:

$$\mathbf{x}_{syn} = (1 - \lambda)\mathbf{x}_i + \lambda \mathbf{x}_j \tag{6}$$

where $\lambda \sim \mathcal{U}[0, 1]$ is a random interpolation factor.

B Optimal 1D Split via Variance Minimization

B.1 Mathematical Formulation

Let $S = \{s_1, s_2, \dots, s_N\}$ be the set of scalar projections of the points in cluster C onto the leading eigenvector $\mathbf{v}$, sorted such that $s_1 \leq s_2 \leq \cdots \leq s_N$.

We seek a split index $k \in \{1, \dots, N-1\}$ that partitions S into a left set $S_L = \{s_1, \dots, s_k\}$ and a right set $S_R = \{s_{k+1}, \dots, s_N\}$ to minimize the total variance:

$$k^* = \text{argmin}_k \left(\sum_{s \in S_L} (s - \mu_L)^2 + \sum_{s \in S_R} (s - \mu_R)^2 \right) \tag{7}$$

where μ_L and μ_R are the means of S_L and S_R respectively.